

JammIA

Chatbot conversazionale su Caravaggio e Caracciolo
e i musei di Napoli

Documentazione tecnica dell'architettura

*Progetto di Natural Language Processing
Corso di Laurea Magistrale*

Manuel Mignogna, Alessia Previdente

Indice

1	Panoramica del sistema	3
1.1	Il flusso di un turno	3
1.2	Struttura del progetto	3
2	Stack tecnologico e librerie	4
2.1	Modello linguistico e orchestrazione	4
2.2	Dati, voce e interfaccia	5
2.3	Selezione del dispositivo e del modello	5
3	La pipeline di ingestion	5
3.1	Estrazione: le query SPARQL	6
3.2	Trasformazione: Extractor	6
3.3	Caricamento: Neo4jLoader e lo schema del grafo	6
4	Il sistema conversazionale: grafo di stato LangGraph	7
4.1	Lo stato: DialogState	7
4.2	I nodi del grafo	8
4.3	Il routing condizionale	8
4.4	Human-in-the-loop e persistenza	9
5	Classificazione e scomposizione della richiesta	9
5.1	Lo schema di output: ModelResponse	9
5.2	Le tre categorie	10
5.3	Un prompt per ogni ruolo	10
5.4	Scelte di prompting	10
5.5	Robustezza	11
6	Il modulo RAG: da linguaggio naturale a Cypher	11
6.1	Configurazione della catena	11
6.2	Progettazione del prompt Cypher	12
6.3	Esecuzione parallela e resilienza	12
7	Sintesi della risposta	12
8	Interfaccia e canale vocale	13
8.1	Interfaccia Gradio	13
8.2	Voce: Whisper e gTTS	13

1 Panoramica del sistema

Il progetto realizza **JammIA**, un assistente conversazionale specializzato sulle opere dei pittori **Michelangelo Merisi da Caravaggio** e **Giovanni Battista Caracciolo** (detto Battistello) e i musei di Napoli che le espongono. L'utente può porre domande sia a voce sia per iscritto, e il sistema risponde con testo e sintesi vocale, attingendo a un grafo di conoscenza costruito a partire da fonti aperte (Wikidata e Wikipedia).

L'architettura si articola in **due sottosistemi indipendenti**, separati nel tempo e nelle responsabilità:

1. **Pipeline di ingestion (offline)**. Interroga Wikidata via SPARQL, arricchisce i dati con le descrizioni testuali di Wikipedia e materializza il tutto in un knowledge graph Neo4j. Viene eseguita una tantum (o quando si vuole rigenerare la base dati) tramite `build_database.py`.
2. **Sistema conversazionale (runtime)**. All'avvio (`app.py`) espone un'interfaccia Gradio. Ogni turno di dialogo passa attraverso un grafo di stato LangGraph che classifica la richiesta, la scompone in sotto-domande, recupera i dati dal grafo tramite RAG Text-to-Cypher e sintetizza una risposta in linguaggio naturale, infine convertita in voce.

Il principio di fondo è la **separazione tra conoscenza e ragionamento**: i fatti risiedono in modo strutturato nel grafo, mentre l'LLM è usato come traduttore (linguaggio naturale → Cypher) e come sintetizzatore (dati → risposta), non come fonte di verità. Questo riduce le allucinazioni e ancora le risposte a dati verificabili.

1.1 Il flusso di un turno

Un turno attraversa il sistema in questo ordine:

1. **Input**. L'utente scrive nel campo di testo oppure parla al microfono; in tal caso Whisper trascrive l'audio (normalizzato in mono float32 nell'intervallo $[-1, 1]$, con lingua forzata all'italiano).
2. **Instradamento**. `ChatController` inoltra la domanda al `DialogManager` con il `thread_id` della sessione. Se il turno precedente si era sospeso con una domanda di chiarimento, il messaggio riprende il grafo dal punto di sospensione (`Command(resume=...)`); altrimenti parte una nuova invocazione del grafo.
3. **Classificazione**. Il primo nodo classifica la richiesta in tre esiti: *chitchat* (il classificatore stesso formula la risposta sociale), *clarification* (il grafo si sospende e chiede all'utente), *query* (la richiesta è scomposta in sotto-domande atomiche auto-contenute, ciascuna marcata `in_scope`).
4. **Retrieval RAG**. Ogni sotto-domanda in ambito è tradotta in Cypher dall'LLM, eseguita su Neo4j (in parallelo tra sotto-domande) e produce righe grezze.
5. **Sintesi**. Un'unica chiamata LLM fonde tutte le righe in una risposta italiana coerente; alle parti fuori ambito è concatenata una nota fissa sui confini del dominio.
6. **Chiusura**. Il turno (domanda + risposta) è accodato alla cronologia; la risposta è sintetizzata in voce con gTTS (se il servizio fallisce, resta il solo testo) e restituita alla UI come testo + audio.

1.2 Struttura del progetto

Il codice è organizzato in package Python per responsabilità:

Package / file	Responsabilità
<code>chatbot/ingestion/</code>	Costruzione del knowledge graph: query SPARQL, estrazione, caricamento in Neo4j.

<code>chatbot/rag/</code>	Catena RAG Text-to-Cypher su Neo4j e prompt di generazione query / sintesi.
<code>chatbot/dialog/</code>	Gestione del dialogo: grafo di stato LangGraph, classificazione, stati, prompt di routing.
<code>chatbot/ui/</code>	Interfaccia Gradio e stato di sessione.
<code>chatbot/speech/</code>	Riconoscimento vocale (Whisper) e sintesi vocale (gTTS).
<code>chatbot/config.py</code>	Configurazione centralizzata: credenziali, modelli, percorsi, costanti.
<code>query/*.psql</code>	Le quattro query SPARQL parametriche (artisti, opere, musei).
<code>app.py</code> , <code>build_database.py</code>	Entry point: avvio UI e popolamento del database.

2 Stack tecnologico e librerie

La scelta delle librerie riflette due priorità: eseguire tutto **in locale** (nessuna dipendenza da API a pagamento per l'LLM) e appoggiarsi ad **astrazioni consolidate** (LangChain/LangGraph) per non reimplementare orchestrazione e integrazione con il grafo.

2.1 Modello linguistico e orchestrazione

Libreria	Ruolo e motivazione
<code>langchain-ollama</code>	LLM locale (<code>ChatOllama</code>) — esegue Gemma via Ollama (o backend compatibili) senza costi né chiavi API: è il provider di default.
<code>langchain-google-genai</code>	LLM cloud (<code>ChatGoogleGenerativeAI</code>) — provider alternativo (Gemini via API key): stessa interfaccia LangChain, selezionabile con <code>LLM_PROVIDER=gemini</code> senza modifiche al codice.
<code>langgraph</code>	Grafo di stato del dialogo — modella il turno come macchina a stati con nodi, edge condizionali e human-in-the-loop (<code>interrupt</code>); include il checkpointer <code>MemorySaver</code> .
<code>MemorySaver</code> (<code>langgraph</code>)	Persistenza dello stato — checkpointer in memoria che conserva lo stato della conversazione tra i turni, indicizzato per <code>thread_id</code> ; adeguato a un'app a processo singolo (lo stato si azzerà al riavvio).
<code>langchain-neo4j</code>	RAG sul grafo — fornisce <code>Neo4jGraph</code> e <code>GraphCypherQAChain</code> (Text-to-Cypher) pronti all'uso.
<code>pydantic</code>	Modelli tipizzati — definisce lo schema dell'output del classificatore (<code>ModelResponse</code>) e lo stato del dialogo (<code>DialogState</code>); l'output JSON del modello è validato con <code>model_validate_json</code> .

2.2 Dati, voce e interfaccia

Libreria	Ruolo e motivazione
<code>sparqlwrapper</code>	Client SPARQL — interroga l'endpoint di Wikidata; gestione di formato JSON e header.
<code>neo4j</code>	Driver del grafo — accesso diretto a Neo4j per la fase di caricamento (MERGE dei nodi/relazioni).
<code>transformers</code> + <code>torch</code> + <code>torchaudio</code>	Speech-to-Text — eseguono Whisper large-v3 in locale per la trascrizione dell'audio.
<code>gtts</code>	Text-to-Speech — sintesi vocale semplice e in italiano della risposta finale.
<code>gradio</code>	Interfaccia web — UI chat con supporto nativo a microfono, audio e animazioni di caricamento.
<code>mlx-lm</code>	Backend Apple Silicon — abilita l'inferenza ottimizzata su GPU Apple (variante <code>-mlx</code> del modello).

2.3 Selezione del dispositivo e del modello

La configurazione (`config.py`) sceglie automaticamente l'acceleratore disponibile (CUDA su NVIDIA/Colab, altrimenti MPS su Apple Silicon, altrimenti CPU) e di conseguenza la variante del modello Gemma (`gemma4:e4b-mlx` su Apple, `gemma:e4b` altrove). Tutte le costanti sensibili (URI e credenziali Neo4j, nomi dei modelli, endpoint, API key) sono lette da variabili d'ambiente con valori di default, così da non essere disseminate nel codice.

La creazione dei chat model è centralizzata nella factory `config.make_llm(temperature)`: `RagChain` e `DialogManager` non conoscono il provider, che si seleziona con `LLM_PROVIDER` (`ollama`, default locale, oppure `gemini` con `GOOGLE_API_KEY` e modello configurabile via `GEMINI_MODEL`). Il resto della pipeline (parsing JSON, grafo di stato, RAG) è indifferente al provider, perché usa la sola interfaccia comune dei chat model LangChain. Con il provider cloud si rinuncia all'esecuzione interamente locale (le domande transitano dai server Google), in cambio di una qualità di classificazione e sintesi sensibilmente superiore.

3 La pipeline di ingestion

L'obiettivo di questa fase è trasformare dati eterogenei e semi-strutturati (Wikidata) e testi discorsivi (Wikipedia) in un grafo pulito e interrogabile. È orchestrata da `pipeline.populate_database()` e si compone di tre stadi: **estrazione**, **trasformazione/arricchimento**, **caricamento** (un classico ETL).

Perché Wikidata e non DBpedia

Come sorgente dei dati strutturati era stata inizialmente valutata **DBpedia**, alternativa naturale a Wikidata e anch'essa interrogabile via SPARQL. In pratica, però, per le opere e gli artisti del dominio DBpedia esponeva proprietà **troppo scarse o del tutto assenti** (date, dimensioni, collocazioni, relazioni), insufficienti a costruire un grafo utile e a produrre risposte di qualità accettabile. Si è quindi scelta **Wikidata**, la cui copertura di entità e proprietà per questo

dominio è nettamente più completa e regolare, e che offre un identificativo stabile (Q-ID) su cui ancorare i nodi. Le descrizioni testuali, invece, sono attinte alle API di Wikipedia (v. stadio di trasformazione), più ricche degli abstract disponibili altrove.

3.1 Estrazione: le query SPARQL

Le quattro query in `query/*.psql` interrogano Wikidata a partire dagli identificatori dei due artisti (`Q42207` per Caravaggio, `Q2519261` per Caracciolo). Sono state scritte con alcune accortezze ricorrenti:

- **Preferenza linguistica con fallback.** Le etichette sono richieste in italiano e, se assenti, in inglese, tramite `COALESCE(?labelIT, ?labelEN, "default")`. Questo evita campi vuoti quando Wikidata non ha la traduzione italiana.
- **Aggregazione per evitare duplicati.** Proprietà multi-valore (soggetti, movimenti, opere notevoli) sono raccolte con `GROUP_CONCAT(DISTINCT ...)`; i valori singoli con `SAMPLE(...)`, così ogni entità produce una sola riga.
- **Fonti alternative unite con UNION.** Il luogo di un'opera è cercato sia come «collocazione» (`P276`) sia come «collezione» (`P195`); l'indirizzo del museo attraverso più proprietà. In questo modo si massimizza la copertura di un grafo di conoscenza notoriamente irregolare.

`SparqlExecutor` carica le query, le esegue con **retry sul rate limit** (HTTP 429, attesa e nuovo tentativo) e **memorizza i risultati in cache** (`cache/sparql_cache.json`). Gli errori non transienti seguono invece una politica *fail-loudly*: l'eccezione risale e interrompe la pipeline, così un fallimento non viene mai salvato in cache come risultato vuoto (che al run successivo maschererebbe per sempre il problema). La cache è cruciale: senza di essa ogni ricostruzione del database rifarebbe decine di richieste a Wikidata, lente e soggette a throttling.

3.2 Trasformazione: Extractor

`Extractor` converte le *binding* SPARQL grezze in dizionari Python puliti e le arricchisce:

- Estrae il Q-ID dagli URI Wikidata, tronca le date alla parte utile (anno o data ISO), e traduce le coordinate dal formato WKT `Point(lon lat)` alla coppia `(lat, lon)`.
- Per le opere, **sostituisce la descrizione sintetica di Wikidata con l'introduzione discorsiva della relativa voce Wikipedia**, recuperata in batch dalle API di Wikipedia IT (`prop=extracts`, solo intro, testo semplice). Anche qui interviene una cache dedicata (`wikipedia_cache.json`) con gestione di redirect e normalizzazioni dei titoli, e throttling per rispettare i limiti del servizio.

Questa scelta è deliberata: le descrizioni di Wikipedia sono molto più ricche e utili per la risposta finale rispetto alle stringhe minimali di Wikidata.

3.3 Caricamento: Neo4jLoader e lo schema del grafo

`Neo4jLoader` è implementato come **context manager** (`__enter__` / `__exit__`) per garantire la chiusura del driver Neo4j anche in presenza di eccezioni.

I nodi sono inseriti con `MERGE ... ON CREATE SET`, idempotente: rieseguire il caricamento non duplica i nodi. Lo schema risultante è il seguente:

Nodo	Proprietà principali
Artista	<code>name</code> , <code>data_nascita</code> , <code>data_morte</code> , <code>luogo_nascita</code> , <code>movimenti</code> , <code>opere_notevoli</code> , <code>wikidataId</code>
Opera	<code>name</code> , <code>anno</code> , <code>altezza</code> , <code>larghezza</code> , <code>tecnica</code> , <code>soggetti</code> , <code>descrizione</code> , <code>tipo</code> , <code>wikidataId</code>
Museo	<code>name</code> , <code>descrizione</code> , <code>indirizzo</code> , <code>sito</code> , <code>telefono</code> , <code>fondazione</code> , <code>latitudine</code> , <code>longitudine</code> , <code>biglietto</code>
Città	<code>name</code>

Le relazioni collegano le entità secondo la semantica del dominio:

```
(Opera) - [:DIPINTA_DA] -> (Artista)
(Opera) - [:ESPOSTA_IN] -> (Museo)
(Museo) - [:SITUATO_IN] -> (Città)
```

L'ordine di caricamento (artisti e musei prima delle opere) non è casuale: `insert_work` cerca artista e museo per creare le relazioni, quindi quei nodi devono già esistere. La ricerca usa `OPTIONAL MATCH` con `FOREACH` condizionale invece di un `MATCH` secco: se un riferimento manca (per esempio un'opera senza museo noto), la relazione corrispondente semplicemente non viene creata, senza troncare in silenzio il resto della query.

Al termine del caricamento (`create_fulltext_indexes`) vengono creati due **indici full-text** Lucene, `operaNameIndex` su `Opera.name` e `museoNameIndex` su `Museo.name`, con `CREATE FULLTEXT INDEX ... IF NOT EXISTS` (idempotente: `DETACH DELETE` non rimuove gli indici). Questi indici abilitano la ricerca **fuzzy** dei titoli in fase di interrogazione (v. modulo RAG).

4 Il sistema conversazionale: grafo di stato LangGraph

Il cuore del runtime è `DialogManager`, che modella un turno di dialogo come un **grafo di stato** (`StateGraph`) di LangGraph. Questa scelta permette di rappresentare esplicitamente il flusso decisionale (classificare, eventualmente chiedere chiarimenti, recuperare e rispondere) con edge condizionali invece che con una cascata di `if` annidati.

4.1 Lo stato: `DialogState`

Lo stato che attraversa il grafo è un modello Pydantic con i campi essenziali del turno:

Campo	Significato
<code>question</code>	La domanda corrente dell'utente (eventualmente arricchita col chiarimento).
<code>sub_questions</code>	Elenco di sotto-domande atomiche, ciascuna con un flag <code>in_scope</code> .
<code>clarification_question</code>	La domanda di chiarimento da porre all'utente, se necessaria.
<code>clarification_attempts</code>	Contatore dei tentativi di chiarimento (per evitare loop infiniti).

<code>history</code>	Gli ultimi turni della conversazione, come lista di <code>Turn</code> (domanda + risposta).
<code>response</code>	La risposta finale (usata anche per il chitchat).

Gli aggiornamenti allo stato usano un `TypedDict` parziale (`DialogStateUpdate`): ogni nodo restituisce solo i campi che modifica, e LangGraph li fonde nello stato. La cronologia è mantenuta **limitata** (ultimi 10 turni salvati, ultimi 3 passati come contesto ai prompt) per contenere la dimensione del prompt; ogni `Turn` è serializzato nel prompt con **ruoli espliciti** («Utente: ...» / «Assistente (tu): ...»), così il modello riconosce che le risposte precedenti, comprese le offerte del tipo «Se vuoi posso darti informazioni anche su...», sono le sue, e può usarle per risolvere riferimenti impliciti e accettazioni («sì», «fallo»).

Il ciclo di vita dello stato è legato al **checkpointer**: a ogni invocazione LangGraph ricarica lo stato salvato per quel `thread_id`, esegue i nodi fondendo i `DialogStateUpdate`, e alla fine (o alla sospensione per `interrupt`) lo ripersiste. La conversazione è quindi interamente ricostruibile dal solo `thread_id`, e la UI non deve trasportare la cronologia a ogni chiamata.

4.2 I nodi del grafo

Nodo	Funzione
<code>USER_PROMPT_CLASSIFICATION</code>	Classifica e scompone la richiesta chiamando l'LLM e validando il JSON prodotto con Pydantic.
<code>USER_INTENT_CLARIFICATION</code>	Sospende il grafo (<code>interrupt</code>) e chiede un chiarimento all'utente.
<code>RESPONSE_GENERATION</code>	Esegue il RAG sulle sotto-domande in ambito e sintetizza la risposta.
<code>HISTORY_UPDATE</code>	Accoda il turno corrente alla cronologia e termina.

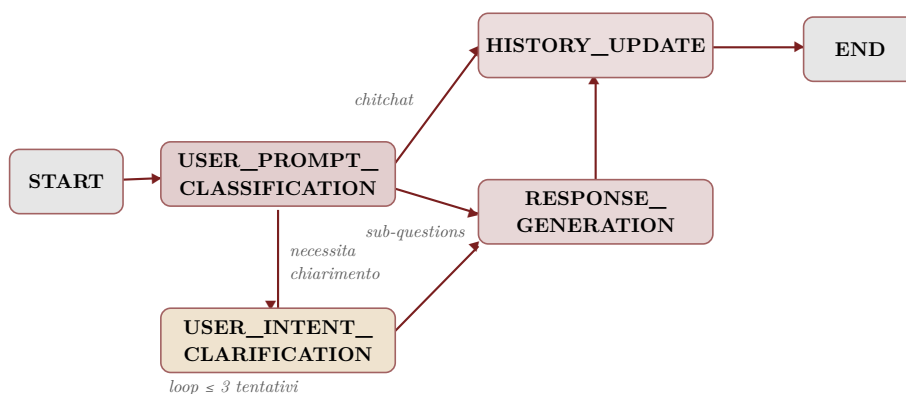


Figura 1: Nodi e archi del grafo di stato del dialogo.

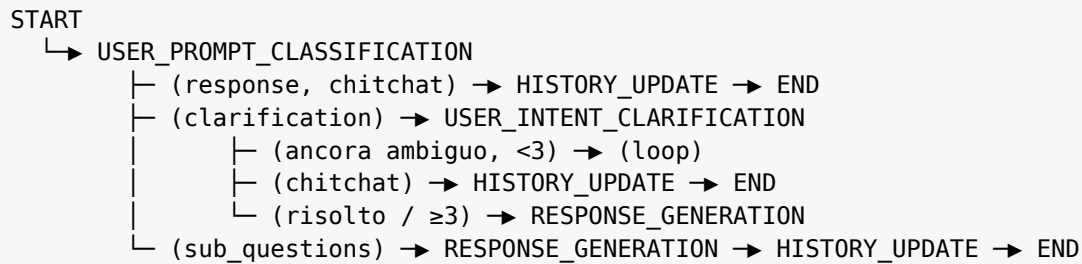
4.3 Il routing condizionale

Dopo la classificazione, una funzione di routing (`_route_after_resolve`) decide il nodo successivo in base a **quali campi dello stato sono valorizzati**, con una precisa priorità:

1. se è presente una `response` (caso chitchat) → si va direttamente all'aggiornamento cronologia, saltando del tutto il grafo;

2. se è presente una `clarification_question` → si va al nodo di chiarimento;
3. se sono presenti `sub_questions` → si va alla generazione della risposta.

Il nodo di chiarimento ha a sua volta un edge condizionale con tre uscite: se dopo la risposta dell'utente la richiesta è ancora ambigua e non si sono superati i **3 tentativi**, si richiede un nuovo chiarimento; se la risposta dell'utente si rivela chitchat (es. «grazie, lascia stare»), la risposta sociale è già pronta nello stato e si passa direttamente all'aggiornamento della cronologia; altrimenti si procede alla generazione. Il limite sui tentativi evita che l'utente resti intrappolato in un ciclo di domande: al terzo fallimento si tenta comunque una risposta con la domanda così com'è.



Codice 1: Flusso del grafo di stato del dialogo.

4.4 Human-in-the-loop e persistenza

Il chiarimento sfrutta il meccanismo `interrupt` di LangGraph: il grafo si **sospende** restituendo la domanda all'interfaccia e riprende esattamente da quel punto quando arriva la risposta dell'utente (`Command(resume=...)`). Perché queste funzioni tra due invocazioni HTTP distinte serve un **checkpoint**: lo stato è serializzato (con `JsonPlusSerializer`, istruito sui modelli Pydantic `Turn` e `SubQuestion`) e indicizzato per `thread_id`, cioè per conversazione. Il checkpoint è un `MemorySaver` in memoria: sufficiente per un'app a processo singolo, con l'ovvio compromesso che le conversazioni si azzerano al riavvio.

Il `thread_id` (UUID) è creato **pigramente al primo turno di ogni sessione** Gradio, non alla costruzione della UI: il valore iniziale di `gr.State` viene infatti valutato una sola volta all'avvio e copiato in ogni sessione, quindi generarlo lì significherebbe far condividere a tutti gli utenti la stessa conversazione. Il pulsante di reset genera un nuovo `thread_id`, ripulendo di fatto la storia.

5 Classificazione e scomposizione della richiesta

Il primo nodo è anche il più delicato: deve decidere **cosa** fare con il messaggio dell'utente prima ancora di toccare il grafo. Chiede all'LLM di produrre un JSON conforme allo schema Pydantic `ModelResponse`, che viene poi validato con `model_validate_json` (la gestione degli output malformati è descritta più avanti, nel paragrafo sulla robustezza). Questo approccio è stato preferito a `with_structured_output` perché la decodifica vincolata (parametro `format` di Ollama) non è garantita da tutti i backend compatibili.

5.1 Lo schema di output: `ModelResponse`

```

class ModelResponse(BaseModel):
    type: Literal["query", "clarification", "chitchat"]
    sub_questions: Optional[list[SubQuestion]] = None # se type == query
    clarification_question: Optional[str] = None # se type == clarification
    response: Optional[str] = None # se type == chitchat
  
```

Ogni tipo usa **un solo campo payload**, senza combinazioni ambigue: questo mantiene la logica di routing pulita (uno switch sul `type` decide quale campo leggere). Per il chitchat il classificatore compila direttamente il campo `response` con la risposta sociale già formulata, così un saluto o un ringraziamento si risolve in **una sola chiamata** all'LLM, senza un secondo passaggio di generazione.

5.2 Le tre categorie

Categoria	Quando scatta e cosa produce
QUERY	La richiesta contiene almeno una domanda su opere/artisti/musei. Viene scomposta in sotto-domande atomiche, ciascuna auto-contenuta e con un flag <code>in_scope</code> .
CLARIFICATION	La richiesta richiederebbe una query ma contiene un riferimento implicito che né il messaggio né la storia risolvono (es. «Chi l'ha dipinta?» senza opera nominata).
CHITCHAT	Messaggi puramente sociali (saluti, ringraziamenti) senza alcuna richiesta di informazioni. La risposta è prodotta dallo stesso classificatore nel campo <code>response</code> .

5.3 Un prompt per ogni ruolo

Il dialogo usa **tre prompt distinti**, ognuno con una sola responsabilità: il **classificatore** (instradamento, scomposizione delle query e, per il chitchat, formulazione diretta della risposta sociale, con la cronologia a disposizione per presentarsi come JammIA solo al primo scambio e non ripetersi), il **prompt Cypher** (generazione della query dal linguaggio naturale) e il **prompt di sintesi** (dai dati alla risposta discorsiva). Tenerli separati evita che le regole di stile di un ruolo «inquinino» gli altri (per esempio, le istruzioni sul tono della risposta non hanno motivo di comparire nel prompt che genera Cypher) e permette di iterare su ciascuno in isolamento.

5.4 Scelte di prompting

Il prompt del classificatore concentra diverse istruzioni non banali:

- **Risoluzione dei riferimenti (coreference).** Ogni sotto-domanda deve essere riscritta in forma completamente auto-contenuta: dimostrativi e pronomi («questi quadri», «lui», «lì») vanno sostituiti con il nome esplicito dell'entità, preso dagli scambi precedenti. Chi legge la sotto-domanda non deve aver bisogno della conversazione per capirla, requisito essenziale perché la generazione Cypher a valle riceve le sotto-domande isolate.
- **Scomposizione di richieste composte.** Una domanda che ne contiene più di una viene spezzata in unità atomiche su un solo argomento. Ciò consente di recuperare e poi ricombinare i dati in modo controllato.
- **Marcatura dell'ambito (`in_scope`) con verifica prima di scartare.** Ogni sotto-domanda è etichettata come dentro o fuori dominio. È marcata fuori ambito (`in_scope: false`) solo quando è generica su un altro artista o tema **senza nominare alcuna opera o museo specifico** («...e quante ne ha fatte Botticelli?»): queste parti non toccano il grafo e ricevono una nota esplicita. Se invece la sotto-domanda nomina un'opera o un museo preciso, resta in ambito **anche se attribuita a un artista non trattato**, perché quel titolo va verificato nel grafo: potrebbe essere in realtà un'opera di Caravaggio o Caracciolo attribuita per errore. In tal caso la sotto-domanda è riscritta per cercare l'opera per nome (senza vincolare l'artista sbagliato) e per chiedere il vero autore, riportando fra parentesi l'attribuzione dell'utente così che la sintesi possa correggerla.
- **Bias verso QUERY in caso di dubbio**, per non rifiutare domande legittime, e disambiguazione esplicita tra i due artisti.

- **Offerte accettate.** Se l'ultima risposta dell'assistente si chiudeva con un'offerta concreta («Se vuoi posso darti informazioni anche su Caracciolo») e l'utente accetta anche genericamente («sì», «fallo», «vai»), la richiesta è sempre QUERY: l'offerta viene riscritta come domanda esplicita, applicando all'argomento offerto la forma dell'ultima richiesta dell'utente.
- **Chiarimento come ultima risorsa.** CLARIFICATION scatta solo se il riferimento è davvero irrisolvibile; è vietato chiedere conferma di una domanda già capita («Vuoi sapere X?» implica che X è già la sotto-domanda). Questa regola, insieme alla precedente, evita catene di chiarimenti superflui.

Coerenza tra prompt e schema

Poiché l'output è vincolato allo schema Pydantic, le chiavi indicate negli esempi del prompt devono coincidere esattamente con i nomi dei campi (`response` , `clarification_question` , `sub_questions`). Un disallineamento (per esempio suggerire una chiave `text` inesistente nello schema) porta il modello a produrre un campo che viene scartato, lasciando il valore atteso a `None` .

5.5 Robustezza

Con modelli locali di piccola taglia l'output JSON può occasionalmente essere malformato o avvolto in testo spurio. La difesa è a due livelli: l'**estrazione tollerante** (`_extract_json`) isola la porzione tra la prima `{` e l'ultima `}` , recuperando i casi più comuni (fence markdown, prefissi come `json` , testo anteposto); per i casi irrecuperabili un **fallback difensivo** intercetta l'errore e degrada verso una richiesta di chiarimento, invece di far crashare l'intero turno. La `temperature` contenuta (0.2) mantiene l'output ragionevolmente stabile senza irrigidirlo del tutto. Analogamente, se il grafo dovesse terminare senza risposta, la UI riceve un messaggio di cortesia invece di un'eccezione.

6 Il modulo RAG: da linguaggio naturale a Cypher

Il recupero delle informazioni segue il paradigma **RAG Text-to-Cypher**: invece di cercare in un indice vettoriale, l'LLM traduce la domanda in una query Cypher che viene eseguita sul grafo Neo4j. `RagChain` incapsula la connessione al grafo, l'LLM e la `GraphCypherQChain` di `LangChain`.

6.1 Configurazione della catena

```
GraphCypherQChain.from_llm(
    llm=self.llm,
    graph=Neo4jGraph(...),
    cypher_prompt=CYPHER_PROMPT,
    return_direct=True,      # restituisce le righe grezze, salta la sintesi interna
    validate_cypher=True,    # corregge la direzione delle relazioni secondo lo
    schema
    allow_dangerous_requests=True,
)
```

Due scelte meritano una spiegazione:

- **return_direct=True** . Di norma `GraphCypherQChain` fa **due** chiamate all'LLM: una per generare il Cypher, una per trasformare i risultati in prosa. Qui la seconda è disattivata: la catena è usata **solo per il retrieval** e restituisce le righe grezze. La trasformazione in risposta discorsiva è centralizzata altrove, in **un'unica** chiamata di sintesi per l'intero turno. Così una domanda composta da tre sotto-domande costa una sola sintesi invece di tre, risparmiando chiamate.

- **validate_cypher=True**. Corregge automaticamente le direzioni delle frecce nelle relazioni per farle combaciare con lo schema, evitando che un arco orientato al contrario restituisca silenziosamente un risultato vuoto.

Lo **schema del grafo viene iniettato automaticamente** nel prompt da **Neo4jGraph**, che lo introspetta: il modello conosce quindi etichette, proprietà e relazioni disponibili quando genera la query.

6.2 Progettazione del prompt Cypher

Il prompt di generazione (**CYPHER_GENERATION_TEMPLATE**) codifica una serie di regole nate dagli errori tipici dei modelli piccoli su Text-to-Cypher:

- **Relazioni bidirezionali senza frecce.** Si impone la sintassi piatta **-[:REL]-** senza **</>**. Un modello che indovina la direzione sbagliata produrrebbe risultati vuoti; togliendo l'orientamento il match funziona a prescindere.
- **Case-insensitive.** Confronti sempre con **toLower(...)** su entrambi i lati, per non fallire su differenze di maiuscole.
- **Ricerca fuzzy di opere e musei.** I titoli non sono cercati né per uguaglianza né con **CONTAINS** sulla frase intera, ma tramite gli indici **full-text** Lucene con operatore fuzzy **~**. La stringa di ricerca è costruita scartando articoli e preposizioni («di»/«della»/«il»...) e rendendo ogni parola di CONTENUTO sia **obbligatoria** (prefisso **+**) sia tollerante ai refusi (suffisso **~**): **CALL db.index.fulltext.queryNodes("operaNameIndex", "+sette~ +opere~**. Questo risolve **+misericordia~**) **YIELD node, score ... ORDER BY score DESC LIMIT 1** due esigenze opposte in un colpo solo: una preposizione diversa («della» invece di «di») o un errore di battitura (**Flagellazione**) combaciano comunque grazie alla distanza di Levenshtein, mentre un titolo con una parola-soggetto inventata (es. **Flagellazione di Babbo Natale**) rende obbligatoria una parola inesistente e restituisce quindi **zero righe**, ricadendo pulitamente nel caso «opera non trovata» invece di essere spacciato per un'opera reale. Il vero autore (**a.name**) è sempre restituito, per verificare e correggere eventuali attribuzioni errate.
- **Alias obbligatori sugli aggregati.** Ogni **count / collect / sum** deve avere un **AS** leggibile, così le righe restituite hanno chiavi comprensibili per la sintesi.
- **Gestione di entità multiple.** Quando la domanda riguarda più opere, si impone un unico pattern con **WHERE ... IN [...]** restituendo ogni entità col proprio valore, anziché legarle tutte allo stesso nodo (errore che spesso produce risultati vuoti).

6.3 Esecuzione parallela e resilienza

Nel nodo di generazione, le sotto-domande **in ambito** sono deduplicate e interrogate **in parallelo** con un **ThreadPoolExecutor** (fino a 5 worker): poiché ogni query è indipendente e passa gran parte del tempo in attesa di rete/DB, il parallelismo abbatte la latenza del turno. Ogni query ha inoltre una **politica di retry** (fino a 3 tentativi) **sulle sole eccezioni** (Cypher non valido, errori di connessione): un risultato vuoto su query riuscita è una risposta legittima («non c'è») e viene restituito subito, senza sprecare chiamate LLM in tentativi inutili.

7 Sintesi della risposta

Recuperate le righe dal grafo, un'unica chiamata all'LLM (**COMBINE_PROMPTS_TEMPLATE**) le fonde in una risposta coerente in italiano. Il prompt di sintesi è fortemente vincolato nello stile, per contrastare le tendenze indesiderate dei modelli generativi:

- risposta **concisa** (3–4 frasi), diretta al dato, senza premesse né inviti finali;
- **divieto di citare la fonte** («secondo il database», «nel mio archivio»): il modello deve rispondere come se conoscesse i fatti;

- **divieto di disclaimer** quando i dati ci sono, e uso di **tutti e soli** i valori presenti (se è una lista, elencarli tutti senza inventarne);
- se i dati di una sotto-domanda sono vuoti, quella informazione è dichiarata non disponibile senza inventare, rispondendo comunque alle altre;
- tono cordiale e diretto, in italiano chiaro, adatto anche alla sintesi vocale;
- **titolo esatto e correzioni**: l'opera va sempre nominata col titolo esatto restituito dai dati (`o.name`), mai con la forma scritta dall'utente; se quest'ultima differiva (preposizione diversa, refuso), la risposta lo segnala con garbo indicando la forma corretta. Analogamente, se i dati mostrano un autore diverso da quello che l'utente attribuiva, la risposta corregge l'attribuzione;
- **verifica del soggetto (rete di sicurezza)**: i titoli inventati sono già scartati a monte dalla query (parole di contenuto obbligatorie, v. modulo RAG), ma come ulteriore difesa la sintesi confronta comunque la parola-soggetto chiesta con il titolo nei dati: se differiscono (es. «Babbo Natale» vs «Cristo») non spaccia il risultato per l'opera richiesta e dichiara che quella richiesta non esiste, eventualmente citando quella simile realmente presente;
- **suggerimento finale vincolato**: la risposta può chiudersi con un'offerta («Se vuoi posso darti informazioni anche su...»), ma solo su entità in ambito, nominate esplicitamente e non ancora approfondite. Il vincolo di concretezza non è cosmetico: un'offerta esplicita è ciò che permette al classificatore, al turno successivo, di risolvere un'accettazione generica («sì», «fallo») senza chiedere chiarimenti.

Alle sotto-domande **fuori ambito** è riservata una nota fissa che ricorda i confini del dominio, concatenata alla risposta in-ambito. In questo modo una richiesta mista riceve una risposta completa e onesta: dati reali dove il grafo li ha, delimitazione esplicita dove no.

Perché sintesi separata dal retrieval

Separare il recupero (una query per sotto-domanda) dalla sintesi (una sola chiamata per turno) è la scelta architetturale che tiene basso il numero di invocazioni dell'LLM e centralizza il controllo dello stile in un unico prompt, invece di disperderlo nella generazione interna della catena.

8 Interfaccia e canale vocale

8.1 Interfaccia Gradio

`ChatController` costruisce l'interfaccia, brandizzata JammIA, con tema Gradio sui toni dell'azzurro Napoli (passato a `launch()`, come richiesto da Gradio 6), e ne espone gli handler. Ogni turno è diviso in **due eventi concatenati** (`step 1 .then(step 2)`): il primo aggiunge subito il messaggio dell'utente alla chat (feedback immediato), il secondo calcola la risposta. Sono supportati due ingressi, testo e microfono, che convergono sullo stesso passo di generazione.

Lo stato di sessione (`SessionState`, in `gr.State`) conserva due informazioni: il `thread_id` della conversazione (creato pigramente al primo turno, v. sopra) e un flag `awaiting_clarification`, che indica se il messaggio successivo va instradato come **risposta a un chiarimento** anziché come nuova domanda. Il pulsante di reset rigenera il `thread_id`, ripulendo di fatto la storia.

8.2 Voce: Whisper e gTTS

Il riconoscimento vocale (`SpeechToText`) usa **Whisper large-v3** tramite la pipeline di `transformers`, forzando la lingua italiana. L'audio (`sample_rate`, `ndarray`) prodotto dal microfono di Gradio arriva come interi (tipicamente int16, eventualmente stereo) e viene **normalizzato** nel formato atteso da Whisper (mono, float32, valori in $[-1, 1]$); senza questa normalizzazione la trascrizione degrada sensibilmente. La sintesi vocale (`TextToSpeech`) usa **gTTS** e scrive ogni

risposta in un **file temporaneo distinto** (niente file condiviso: eviterebbe race condition tra sessioni concorrenti e audio obsoleto in cache al browser). La sintesi è **tollerante ai guasti**: se il servizio TTS fallisce, il turno mostra comunque la risposta testuale, semplicemente senza audio.